Mo Tu We Th Fr Sa Su

OS

Date 12/10/26.

Components of OS



User space Kernel

User space:

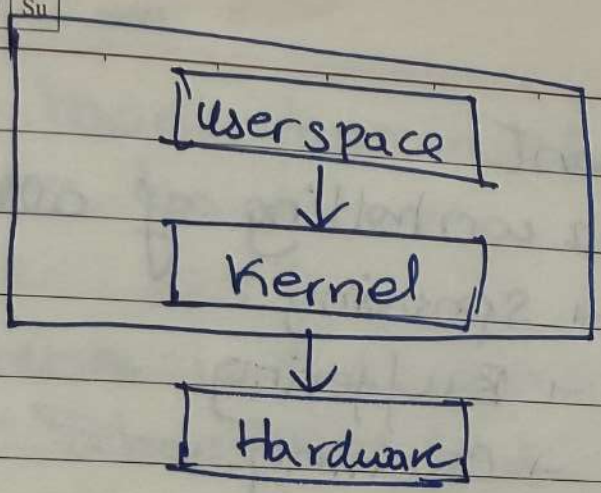
- User interactable space which doesn't have any hardware access.
- User can interact with the Apps via:
  - CLI (Command line interface)
  - GUI (Graphical user Interface)
- CLI is completely made of the Terminal / shell commands
  - Ex) File creation → mkdir
    open file → cd.
- GUI is user interactable in a easy way.
  - Ex) To create file → Select New file
    ↓
    Create file.
- User space interacts with the kernel.

Kernel:

- Heart of OS.
- Responsible for performing actions instructed by the user through user space.

Mo Tu We Th Fr Sa Su

Date / /



⇒ OS

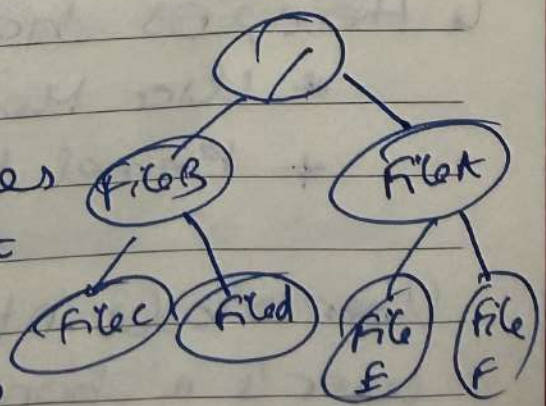
Functions of Kernel:

(i) Process Management:
- Process creation, Termination
- Process syncing
- Process communication
- Process & Thread scheduling.

(ii) Memory Management:
- Allocate & Deallocate Memory
- Free space Management.

(iii) File Management:
- Create / delete files
- Directory Management

Ex) Linux has a / root file under which all directories are branched



Mo Tu We Th Fr Sa Su

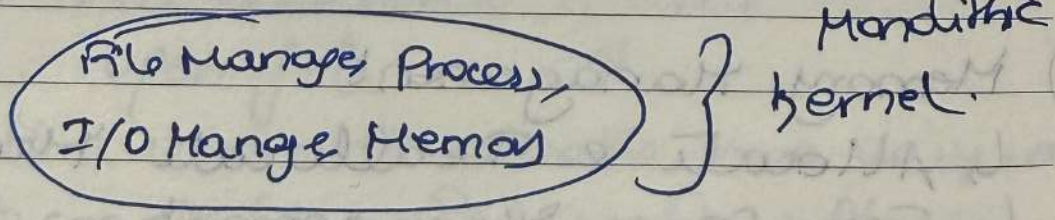
Date / /

(iv) I/O Management
↳ Manage & controlling of all I/O Devices
↳ Handles → Spooling
→ Buffering
→ Caching.

Types of Kernels:

(i) Monolithic Kernel: Ex) Linux, UNIX, MS/DOS
=

↳ Does all the 4 processes / has all 4 components of Kernel within a single Bulky Kernel.



↳ Here, OS has 2 modes:
* User Mode
* Kernel Mode

User mode interacts more g with the user's actions.

Mo Tu We Th Fr Sa Su

Date / /

kernel mode has a closer relationship with the hardware.

↳ OS switches in between one mode to another when needed.

↳ Ex) We create a file using (user mode)
CLI
↓
Kernel's file management (kernel mode)
component has to create & manage file
↓
Again switches back to user mode

↳ Software Interrupt:
This helps in the switch in b/w the user mode & kernel mode.

↳ Major issue → one large kernel, so if one component fails, all others will also fail.

Mo Tu We Th Fr Sa Su

Date / /

(ii) Microkernel:

↳ We can split the kernel components within user space & Kernel space,
ex) our important components such as Process & Memory → Kernel space
File & IO → User space.

↳ Now, communication needs to happen in b/w this User space & Kernel space. This could decrease performance Drastically.

↳ But, it could be more Reliable since important components are kept in ~~another~~ Kernel space.

How does a process running in user space communicate with a process running in Kernel space?

↳ This process of communication happens through:

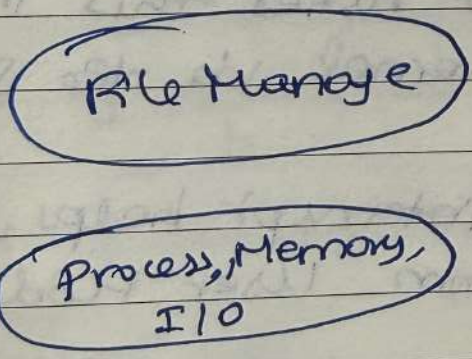
Inter Process communication (IPC)

Mo Tu We Th Fr Sa Su

Date / /

It can happen via:
* Message Passing (or)
* Shared Memory.

(iii) Shared kernel => Hybrid kernel
↳ combined approach.
↳ It puts the File Management alone inside the User space. All others inside the Kernel space.



↳ Ex) MacOS, Windows 7+.

Other two types (Not so imp):
→ Nano Kernel => Extremely small, provides only basic Mechanisms.
→ Exo Kernel => Give Apps more direct control over hardware Resources while also protecting them

Mo Tu We Th Fr Sa Su

Date / /

System Calls

↳ System call / trap is what controls,

What action do we need to perform from user space in the kernel.

Ex) Create a file:

(i) We use mkdir in CLI (user mode)

(ii) System call takes this instruction to the kernel via the System call interface.

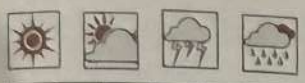
(iii) Software interrupt helps transfering the OS from User Mode to Kernel Mode.

(iv) Kernel uses the file management component to create & manage this file.

Therefore, How apps interact with kernel?

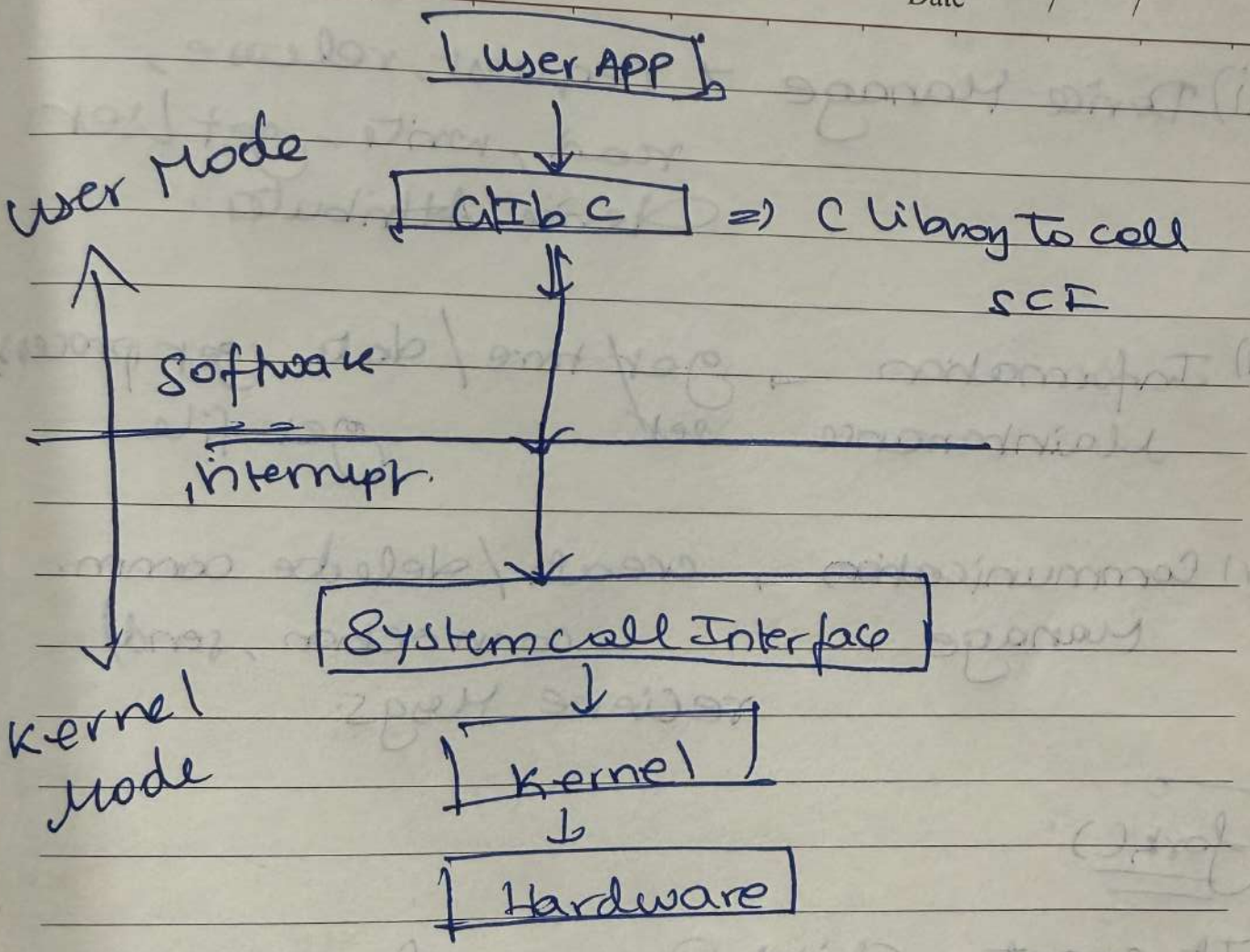
⇓

System calls.



Mo Tu We Th Fr Sa Su

Date / /



Types of system calls:

(i) Process control → end, abort, ~~load~~ load, execute, create/terminate process, wait event, allocate/free memory.

(ii) File Manage → create, read, write, open, close.

Mo Tu We Th Fr Sa Su

Date / /

(iii) Device Manage → request, release, read, write, get/set Device Attributes.

(iv) Information Maintanence → get/set time/date, get process, get file.

(v) Communication Manage → create/delete comm. connection, send/ reciere Msgs.

fork():

It creates child Processes from parent Processes.

Chmod():
Set mode of file → Read/write

Chown:
Change ownership.